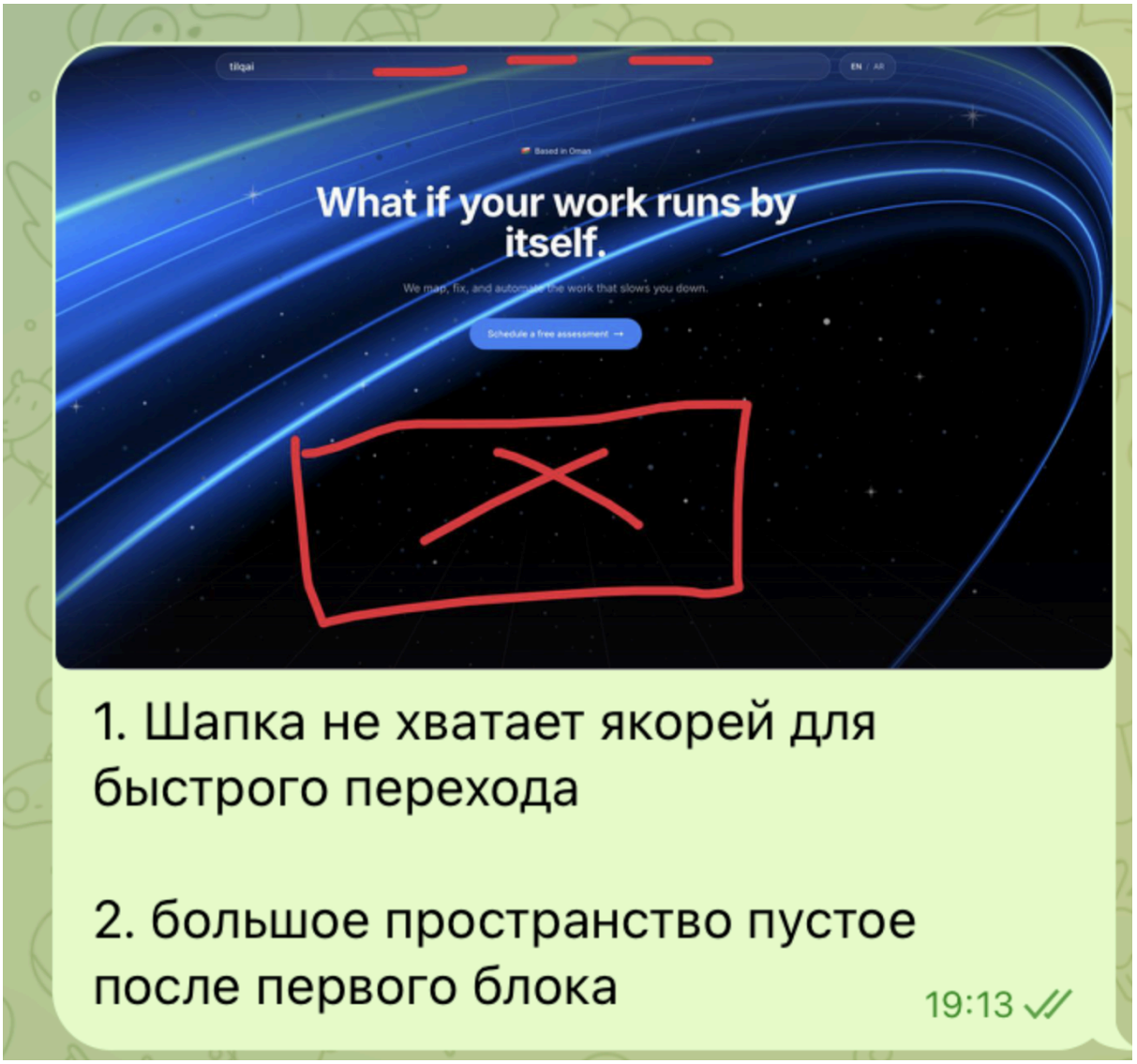
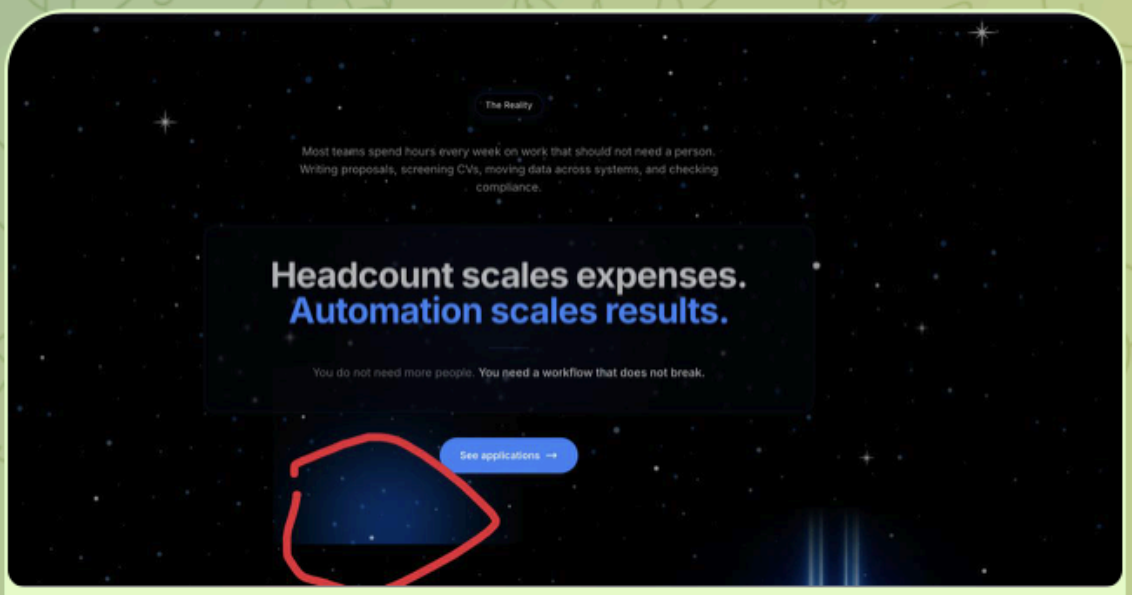
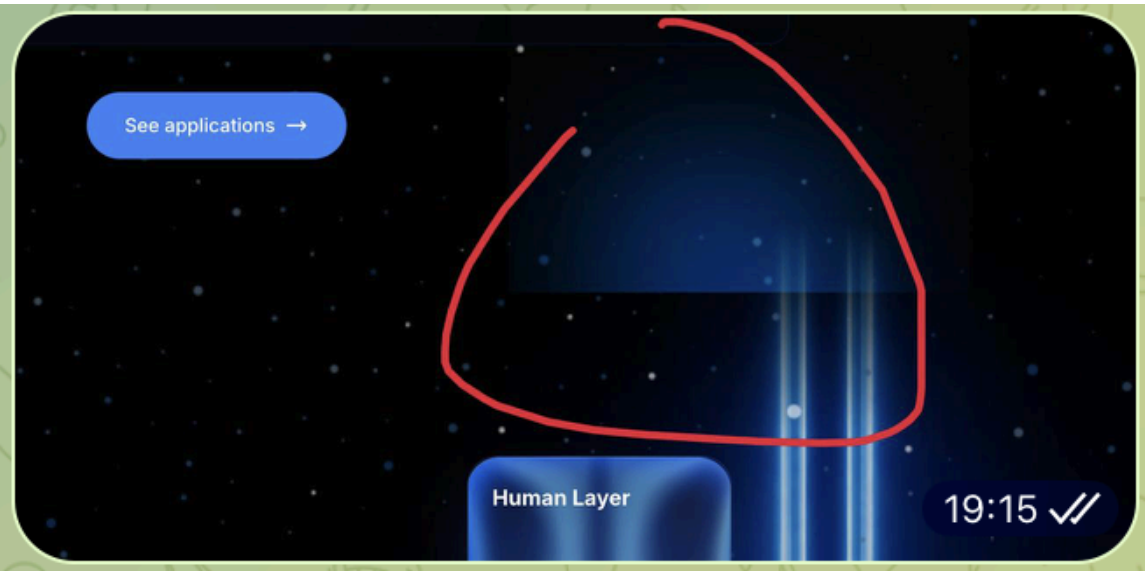

tiqai
EN / AR
Based in Oman
What if your work runs by itself.
We map, fix, and automate the work that slows you down.
Schedule a free assessment →
1. Шапка не хватает якорей для быстрого перехода
2. большое пространство пустое после первого блока
19:13

ПРАВКИ ЛЕНД

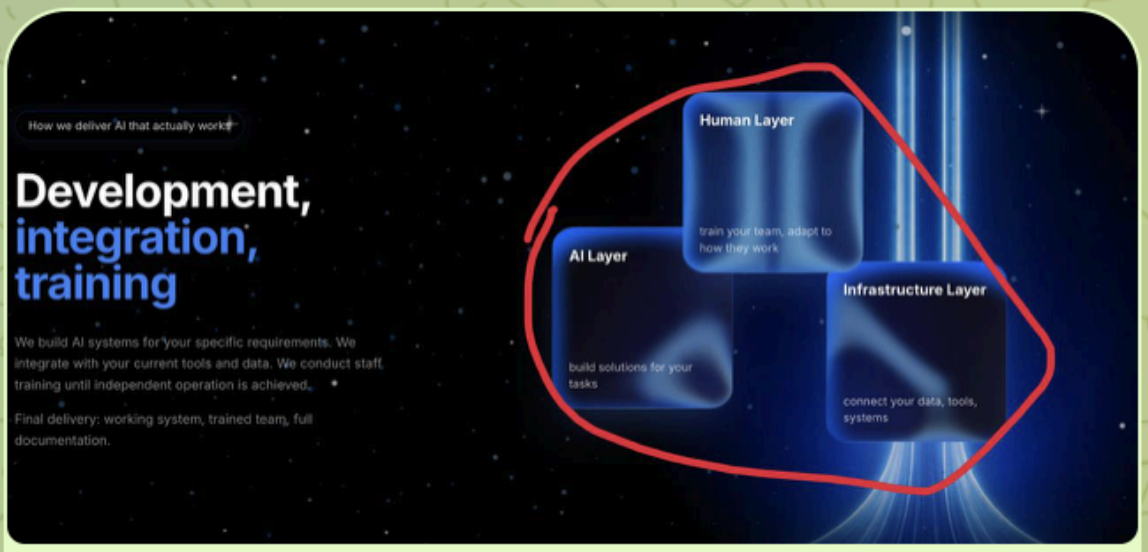


3. вот этот разрыв не очень хорошо смотрится. как будто склейка видео блоков

19:14

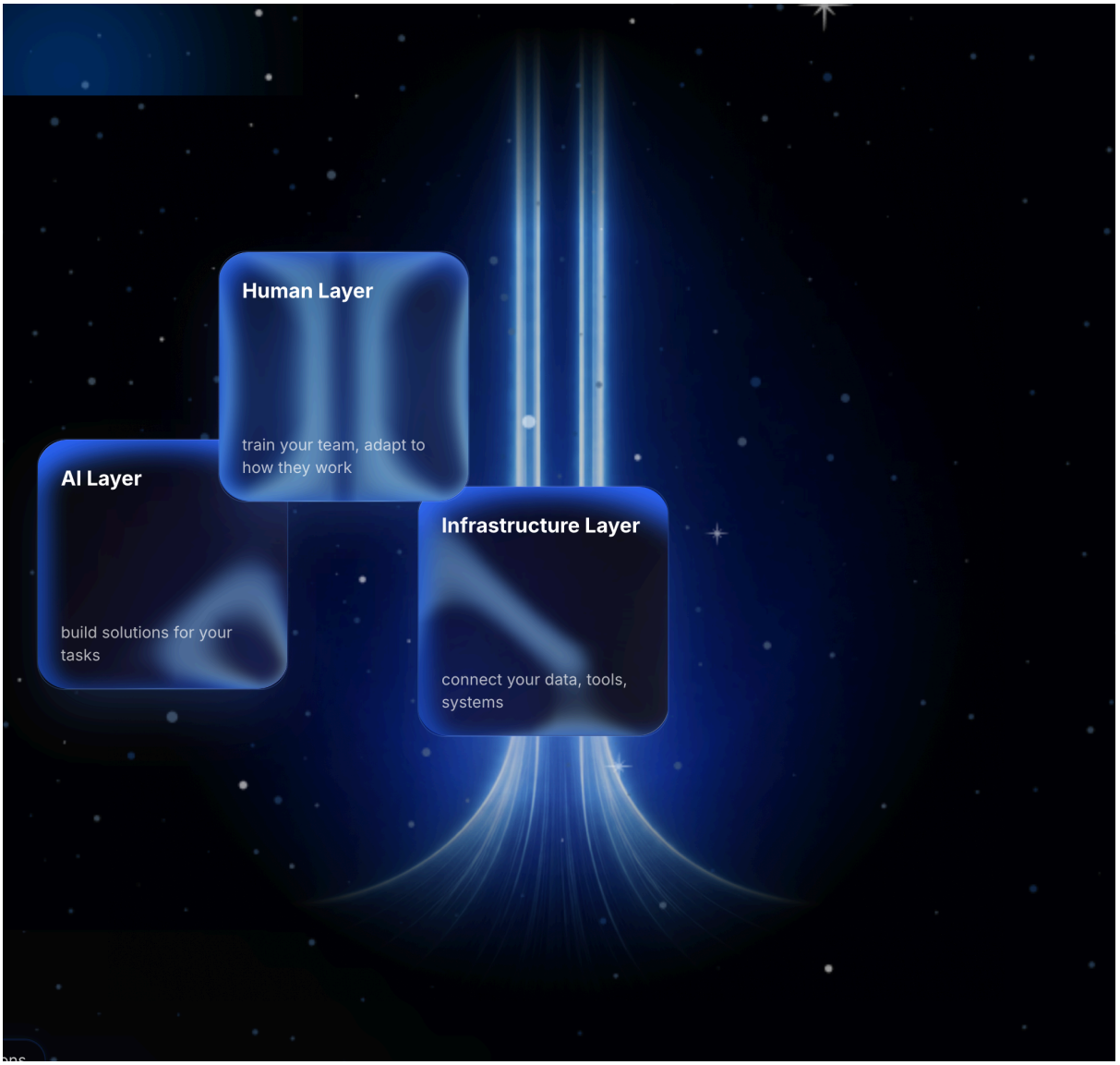


лучше убрать это свечение 19:15



было бы хорошо если бы при наведении они немного двигались как 3д фигуры

19:15



вставить анимацию:
команда терминал: npm install three

код:

```
import LaserFlow from './LaserFlow';
import { useRef } from 'react';

// NOTE: You can also adjust the variables in the shader for super detailed customization

// Basic Usage
<div style={{ height: '500px', position: 'relative', overflow: 'hidden' }}>
  <LaserFlow />
</div>

// Image Example Interactive Reveal Effect
function LaserFlowBoxExample() {
  const revealImgRef = useRef(null);
```

```
return (
  <div
    style={{
      height: '800px',
      position: 'relative',
      overflow: 'hidden',
      backgroundColor: '#060010'
    }}
    onMouseMove={(e) => {
      const rect = e.currentTarget.getBoundingClientRect();
      const x = e.clientX - rect.left;
      const y = e.clientY - rect.top;
      const el = revealImgRef.current;
      if (el) {
        el.style.setProperty('--mx', `${x}px`);
        el.style.setProperty('--my', `${y + rect.height * 0.5}px`);
      }
    }}
    onMouseLeave={() => {
      const el = revealImgRef.current;
      if (el) {
        el.style.setProperty('--mx', '-9999px');
        el.style.setProperty('--my', '-9999px');
      }
    }}
  >
    <LaserFlow
      horizontalBeamOffset={0.1}
      verticalBeamOffset={0.0}
      color="#405ef2"
    />

    <div style={{
      position: 'absolute',
      top: '50%',
      left: '50%',
      transform: 'translateX(-50%)',
      width: '86%',
      height: '60%',
      backgroundColor: '#060010',
      borderRadius: '20px',
      border: '2px solid #FF79C6',
      display: 'flex',
      alignItems: 'center',
      justifyContent: 'center',
      color: 'white',
      fontSize: '2rem',
      zIndex: 6
```

```
      }}>
        {/* Your content here */}
      </div>

      <img
        ref={revealImgRef}
        src="/path/to/image.jpg"
        alt="Reveal effect"
        style={{
          position: 'absolute',
          width: '100%',
          top: '-50%',
          zIndex: 5,
          mixBlendMode: 'lighten',
          opacity: 0.3,
          pointerEvents: 'none',
          '--mx': '-9999px',
          '--my': '-9999px',
          WebkitMaskImage: 'radial-gradient(circle at var(--mx) var(--my), rgba(255,255,255,1)
0px, rgba(255,255,255,0.95) 60px, rgba(255,255,255,0.6) 120px, rgba(255,255,255,0.25)
180px, rgba(255,255,255,0) 240px)',
          maskImage: 'radial-gradient(circle at var(--mx) var(--my), rgba(255,255,255,1) 0px,
rgba(255,255,255,0.95) 60px, rgba(255,255,255,0.6) 120px, rgba(255,255,255,0.25) 180px,
rgba(255,255,255,0) 240px)',
          WebkitMaskRepeat: 'no-repeat',
          maskRepeat: 'no-repeat'
        }}
  horizontalSizing={0.5}
  verticalSizing={2}
  wispDensity={1}
  wispSpeed={15}
  wispIntensity={5}
  flowSpeed={0.35}
  flowStrength={0.25}
  fogIntensity={0.45}
  fogScale={0.3}
  fogFallSpeed={0.6}
  decay={1.1}
  falloffStart={1.2}
/>
    </div>
  );
}

import { useEffect, useRef } from 'react';
import * as THREE from 'three';
import './LaserFlow.css';
```

```
const VERT = `
precision highp float;
attribute vec3 position;
void main(){
  gl_Position = vec4(position, 1.0);
}
`;

const FRAG = `
#ifdef GL_ES
#extension GL_OES_standard_derivatives : enable
#endif
precision highp float;
precision mediump int;

uniform float iTime;
uniform vec3 iResolution;
uniform vec4 iMouse;
uniform float uWispDensity;
uniform float uTiltScale;
uniform float uFlowTime;
uniform float uFogTime;
uniform float uBeamXFrac;
uniform float uBeamYFrac;
uniform float uFlowSpeed;
uniform float uVLenFactor;
uniform float uHLenFactor;
uniform float uFogIntensity;
uniform float uFogScale;
uniform float uWSpeed;
uniform float uWIntensity;
uniform float uFlowStrength;
uniform float uDecay;
uniform float uFalloffStart;
uniform float uFogFallSpeed;
uniform vec3 uColor;
uniform float uFade;

// Core beam/flare shaping and dynamics
#define PI 3.14159265359
#define TWO_PI 6.28318530718
#define EPS 1e-6
#define EDGE_SOFT (DT_LOCAL*4.0)
#define DT_LOCAL 0.0038
#define TAP_RADIUS 6
#define R_H 150.0
#define R_V 150.0
#define FLARE_HEIGHT 16.0
```

```
#define FLARE_AMOUNT 8.0
#define FLARE_EXP 2.0
#define TOP_FADE_START 0.1
#define TOP_FADE_EXP 1.0
#define FLOW_PERIOD 0.5
#define FLOW_SHARPNESS 1.5

// Wisps (animated micro-streaks) that travel along the beam
#define W_BASE_X 1.5
#define W_LAYER_GAP 0.25
#define W_LANES 10
#define W_SIDE_DECAY 0.5
#define W_HALF 0.01
#define W_AA 0.15
#define W_CELL 20.0
#define W_SEG_MIN 0.01
#define W_SEG_MAX 0.55
#define W_CURVE_AMOUNT 15.0
#define W_CURVE_RANGE (FLARE_HEIGHT - 3.0)
#define W_BOTTOM_EXP 10.0

// Volumetric fog controls
#define FOG_ON 1
#define FOG_CONTRAST 1.2
#define FOG_SPEED_U 0.1
#define FOG_SPEED_V -0.1
#define FOG_OCTAVES 5
#define FOG_BOTTOM_BIAS 0.8
#define FOG_TILT_TO_MOUSE 0.05
#define FOG_TILT_DEADZONE 0.01
#define FOG_TILT_MAX_X 0.35
#define FOG_TILT_SHAPE 1.5
#define FOG_BEAM_MIN 0.0
#define FOG_BEAM_MAX 0.75
#define FOG_MASK_GAMMA 0.5
#define FOG_EXPAND_SHAPE 12.2
#define FOG_EDGE_MIX 0.5

// Horizontal vignette for the fog volume
#define HFOG_EDGE_START 0.20
#define HFOG_EDGE_END 0.98
#define HFOG_EDGE_GAMMA 1.4
#define HFOG_Y_RADIUS 25.0
#define HFOG_Y_SOFT 60.0

// Beam extents and edge masking
#define EDGE_X0 0.22
#define EDGE_X1 0.995
```

```
#define EDGE_X_GAMMA 1.25
#define EDGE_LUMA_T0 0.0
#define EDGE_LUMA_T1 2.0
#define DITHER_STRENGTH 1.0

   float g(float x){return x<=0.00031308?12.92*x:1.055*pow(x,1.0/2.4)-0.055;}
   float bs(vec2 p,vec2 q,float powr){
      float d=distance(p,q),f=powr*uFalloffStart,r=(f*f)/(d*d+EPS);
      return powr*min(1.0,r);
   }
   float bsa(vec2 p,vec2 q,float powr,vec2 s){
      vec2 d=p-q; float
dd=(d.x*d.x)/(s.x*s.x)+(d.y*d.y)/(s.y*s.y),f=powr*uFalloffStart,r=(f*f)/(dd+EPS);
      return powr*min(1.0,r);
   }
   float tri01(float x){float f=fract(x);return 1.0-abs(f*2.0-1.0);}
   float tauWf(float t,float tmin,float tmax){float
a=smoothstep(tmin,tmin+EDGE_SOFT,t),b=1.0-smoothstep(tmax-EDGE_SOFT,tmax,t);retur
n max(0.0,a*b);}
   float h21(vec2 p){p=fract(p*vec2(123.34,456.21));p+=dot(p,p+34.123);return
fract(p.x*p.y);}
   float vnoise(vec2 p){
      vec2 i=floor(p),f=fract(p);
      float a=h21(i),b=h21(i+vec2(1,0)),c=h21(i+vec2(0,1)),d=h21(i+vec2(1,1));
      vec2 u=f*f*(3.0-2.0*f);
      return mix(mix(a,b,u.x),mix(c,d,u.x),u.y);
   }
   float fbm2(vec2 p){
      float v=0.0,amp=0.6; mat2 m=mat2(0.86,0.5,-0.5,0.86);
      for(int i=0;i<FOG_OCTAVES;++i){v+=amp*vnoise(p); p=m*p*2.03+17.1; amp*=0.52;}
      return v;
   }
   float rGate(float x,float l){float
a=smoothstep(0.0,W_AA,x),b=1.0-smoothstep(l,l+W_AA,x);return max(0.0,a*b);}
   float flareY(float y){float
t=clamp(1.0-(clamp(y,0.0,FLARE_HEIGHT)/max(FLARE_HEIGHT,EPS)),0.0,1.0);return
pow(t,FLARE_EXP);}

   float vWisps(vec2 uv,float topF){
   float y=uv.y,yf=(y+uFlowTime*uWSpeed)/W_CELL;
   float dRaw=clamp(uWispDensity,0.0,2.0),d=dRaw<=0.0?1.0:dRaw;
   float lanesF=floor(float(W_LANES)*min(d,1.0)+0.5); // WebGL1-safe
   int lanes=int(max(1.0,lanesF));
   float sp=min(d,1.0),ep=max(d-1.0,0.0);
   float
fm=flareY(max(y,0.0)),rm=clamp(1.0-(y/max(W_CURVE_RANGE,EPS)),0.0,1.0),cm=fm*rm;
   const float G=0.05; float xS=1.0+(FLARE_AMOUNT*W_CURVE_AMOUNT*G)*cm;
   float sPix=clamp(y/R_V,0.0,1.0),bGain=pow(1.0-sPix,W_BOTTOM_EXP),sum=0.0;
```

```
    for(int s=0;s<2;++s){
        float sgn=s==0?-1.0:1.0;
        for(int i=0;i<W_LANES;++i){
            if(i>=lanes) break;
            float off=W_BASE_X+float(i)*W_LAYER_GAP,xc=sgn*(off*xS);
            float
dx=abs(uv.x-xc),lat=1.0-smoothstep(W_HALF,W_HALF+W_AA,dx),amp=exp(-off*W_SIDE_
DECAY);
            float seed=h21(vec2(off,sgn*17.0)),yf2=yf+seed*7.0,ci=floor(yf2),fy=fract(yf2);
            float seg=mix(W_SEG_MIN,W_SEG_MAX,h21(vec2(ci,off*2.3)));
            float spR=h21(vec2(ci,off+sgn*31.0)),seg1=rGate(fy,seg)*step(spR,sp);
            if(ep>0.0){float spR2=h21(vec2(ci*3.1+7.0,off*5.3+sgn*13.0)); float f2=fract(fy+0.5);
seg1+=rGate(f2,seg*0.9)*step(spR2,ep);}
            sum+=amp*lat*seg1;
        }
    }
    float span=smoothstep(-3.0,0.0,y)*(1.0-smoothstep(R_V-6.0,R_V,y));
    return uWIntensity*sum*topF*bGain*span;
}

void mainImage(out vec4 fc,in vec2 frag){
    vec2 C=iResolution.xy*.5; float invW=1.0/max(C.x,1.0);
    float sc=512.0/iResolution.x*.4;
    vec2
uv=(frag-C)*sc,off=vec2(uBeamXFrac*iResolution.x*sc,uBeamYFrac*iResolution.y*sc);
    vec2 uvc = uv - off;
    float a=0.0,b=0.0;
    float basePhase=1.5*PI+uDecay*.5; float tauMin=basePhase-uDecay; float
tauMax=basePhase;
    float
cx=clamp(uvc.x/(R_H*uHLenFactor),-1.0,1.0),tH=clamp(TWO_PI-acos(cx),tauMin,tauMax);
    for(int k=-TAP_RADIUS;k<=TAP_RADIUS;++k){
        float tu=tH+float(k)*DT_LOCAL,wt=tauWf(tu,tauMin,tauMax); if(wt<=0.0) continue;
        float
spd=max(abs(sin(tu)),0.02),u=clamp((basePhase-tu)/max(uDecay,EPS),0.0,1.0),env=pow(1.
0-abs(u*2.0-1.0),0.8);
        vec2 p=vec2((R_H*uHLenFactor)*cos(tu),0.0);
        a+=wt*bs(uvc,p,env*spd);
    }
    float
yPix=uvc.y,cy=clamp(-yPix/(R_V*uVLenFactor),-1.0,1.0),tV=clamp(TWO_PI-acos(cy),tauMin
,tauMax);
    for(int k=-TAP_RADIUS;k<=TAP_RADIUS;++k){
        float tu=tV+float(k)*DT_LOCAL,wt=tauWf(tu,tauMin,tauMax); if(wt<=0.0) continue;
        float yb=(-R_V)*cos(tu),s=clamp(yb/R_V,0.0,1.0),spd=max(abs(sin(tu)),0.02);
        float env=pow(1.0-s,0.6)*spd;
        float cap=1.0-smoothstep(TOP_FADE_START,1.0,s); cap=pow(cap,TOP_FADE_EXP);
env*=cap;
```

```
        float ph=s/max(FLOW_PERIOD,EPS)+uFlowTime*uFlowSpeed;
        float fl=pow(tri01(ph),FLOW_SHARPNESS);
        env*=mix(1.0-uFlowStrength,1.0,fl);
        float
yp=(-R_V*uVLenFactor)*cos(tu),m=pow(smoothstep(FLARE_HEIGHT,0.0,yp),FLARE_EXP),
wx=1.0+FLARE_AMOUNT*m;
        vec2 sig=vec2(wx,1.0),p=vec2(0.0,yp);
        float mask=step(0.0,yp);
        b+=wt*bsa(uvc,p,mask*env,sig);
    }
    float
sPix=clamp(yPix/R_V,0.0,1.0),topA=pow(1.0-smoothstep(TOP_FADE_START,1.0,sPix),TOP
_FADE_EXP);
    float L=a+b*topA;
    float w=vWisps(vec2(uvc.x,yPix),topA);
    float fog=0.0;
#if FOG_ON
    vec2 fuv=uvc*uFogScale;
    float mAct=step(1.0,length(iMouse.xy)),nx=((iMouse.x-C.x)*invW)*mAct;
    float ax = abs(nx);
    float stMag = mix(ax, pow(ax, FOG_TILT_SHAPE), 0.35);
    float st = sign(nx) * stMag * uTiltScale;
    st = clamp(st, -FOG_TILT_MAX_X, FOG_TILT_MAX_X);
    vec2 dir=normalize(vec2(st,1.0));
    fuv+=uFogTime*uFogFallSpeed*dir;
    vec2 prp=vec2(-dir.y,dir.x);
    fuv+=prp*(0.08*sin(dot(uvc,prp)*0.08+uFogTime*0.9));
    float n=fbm2(fuv+vec2(fbm2(fuv+vec2(7.3,2.1)),fbm2(fuv+vec2(-3.7,5.9)))*0.6);
    n=pow(clamp(n,0.0,1.0),FOG_CONTRAST);
    float pixW = 1.0 / max(iResolution.y, 1.0);
#ifdef GL_OES_standard_derivatives
    float wL = max(fwidth(L), pixW);
#else
    float wL = pixW;
#endif
    float m0=pow(smoothstep(FOG_BEAM_MIN - wL, FOG_BEAM_MAX + wL,
L),FOG_MASK_GAMMA);
    float bm=1.0-pow(1.0-m0,FOG_EXPAND_SHAPE);
bm=mix(bm*m0,bm,FOG_EDGE_MIX);
    float
yP=1.0-smoothstep(HFOG_Y_RADIUS,HFOG_Y_RADIUS+HFOG_Y_SOFT,abs(yPix));
    float
nxF=abs((frag.x-C.x)*invW),hE=1.0-smoothstep(HFOG_EDGE_START,HFOG_EDGE_END,
nxF); hE=pow(clamp(hE,0.0,1.0),HFOG_EDGE_GAMMA);
    float hW=mix(1.0,hE,clamp(yP,0.0,1.0));
    float bBias=mix(1.0,1.0-sPix,FOG_BOTTOM_BIAS);
    float browserFogIntensity = uFogIntensity;
    browserFogIntensity *= 1.8;
```

```
    float radialFade = 1.0 - smoothstep(0.0, 0.7, length(uvc) / 120.0);
    float safariFog = n * browserFogIntensity * bBias * bm * hW * radialFade;
    fog = safariFog;
#endif
    float LF=L+fog;
    float dith=(h21(frag)-0.5)*(DITHER_STRENGTH/255.0);
    float tone=g(LF+w);
    vec3 col=tone*uColor+dith;
    float alpha=clamp(g(L+w*0.6)+dith*0.6,0.0,1.0);
    float
nxE=abs((frag.x-C.x)*invW),xF=pow(clamp(1.0-smoothstep(EDGE_X0,EDGE_X1,nxE),0.0,1
.0),EDGE_X_GAMMA);
    float
scene=LF+max(0.0,w)*0.5,hi=smoothstep(EDGE_LUMA_T0,EDGE_LUMA_T1,scene);
    float eM=mix(xF,1.0,hi);
    col*=eM; alpha*=eM;
    col*=uFade; alpha*=uFade;
    fc=vec4(col,alpha);
}

void main(){
  vec4 fc;
  mainImage(fc, gl_FragCoord.xy);
  gl_FragColor = fc;
}
`;

export const LaserFlow = ({
  className,
  style,
  wispDensity = 1,
  dpr,
  mouseSmoothTime = 0.0,
  mouseTiltStrength = 0.01,
  horizontalBeamOffset = 0.1,
  verticalBeamOffset = 0.0,
  flowSpeed = 0.35,
  verticalSizing = 2.0,
  horizontalSizing = 0.5,
  fogIntensity = 0.45,
  fogScale = 0.3,
  wispSpeed = 15.0,
  wispIntensity = 5.0,
  flowStrength = 0.25,
  decay = 1.1,
  falloffStart = 1.2,
  fogFallSpeed = 0.6,
  color = '#FF79C6'
```

```
}) => {
  const mountRef = useRef(null);
  const rendererRef = useRef(null);
  const uniformsRef = useRef(null);
  const hasFadedRef = useRef(false);
  const rectRef = useRef(null);
  const baseDprRef = useRef(1);
  const currentDprRef = useRef(1);
  const lastSizeRef = useRef({ width: 0, height: 0, dpr: 0 });
  const fpsSamplesRef = useRef([]);
  const lastFpsCheckRef = useRef(performance.now());
  const emaDtRef = useRef(16.7);
  const pausedRef = useRef(false);
  const inViewRef = useRef(true);

  const hexToRGB = hex => {
    let c = hex.trim();
    if (c[0] === '#') c = c.slice(1);
    if (c.length === 3)
      c = c
        .split('')
        .map(x => x + x)
        .join('');
    const n = parseInt(c, 16) || 0xffffff;
    return { r: ((n >> 16) & 255) / 255, g: ((n >> 8) & 255) / 255, b: (n & 255) / 255 };
  };

  useEffect(() => {
    const mount = mountRef.current;
    const renderer = new THREE.WebGLRenderer({
      antialias: false,
      alpha: false,
      depth: false,
      stencil: false,
      powerPreference: 'high-performance',
      premultipliedAlpha: false,
      preserveDrawingBuffer: false,
      failIfMajorPerformanceCaveat: false,
      logarithmicDepthBuffer: false
    });
    rendererRef.current = renderer;

    baseDprRef.current = Math.min(dpr ?? (window.devicePixelRatio || 1), 2);
    currentDprRef.current = baseDprRef.current;

    renderer.setPixelRatio(currentDprRef.current);
    renderer.shadowMap.enabled = false;
    renderer.outputColorSpace = THREE.SRGBColorSpace;
```

```
    renderer.setClearColor(0x000000, 1);
    const canvas = renderer.domElement;
    canvas.style.width = '100%';
    canvas.style.height = '100%';
    canvas.style.display = 'block';
    mount.appendChild(canvas);

    const scene = new THREE.Scene();
    const camera = new THREE.OrthographicCamera(-1, 1, 1, -1, 0, 1);

    const geometry = new THREE.BufferGeometry();
    geometry.setAttribute('position', new THREE.BufferAttribute(new Float32Array([-1, -1, 0, 3,
-1, 0, -1, 3, 0]), 3));

    const uniforms = {
      iTime: { value: 0 },
      iResolution: { value: new THREE.Vector3(1, 1, 1) },
      iMouse: { value: new THREE.Vector4(0, 0, 0, 0) },
      uWispDensity: { value: wispDensity },
      uTiltScale: { value: mouseTiltStrength },
      uFlowTime: { value: 0 },
      uFogTime: { value: 0 },
      uBeamXFrac: { value: horizontalBeamOffset },
      uBeamYFrac: { value: verticalBeamOffset },
      uFlowSpeed: { value: flowSpeed },
      uVLenFactor: { value: verticalSizing },
      uHLenFactor: { value: horizontalSizing },
      uFogIntensity: { value: fogIntensity },
      uFogScale: { value: fogScale },
      uWSpeed: { value: wispSpeed },
      uWIntensity: { value: wispIntensity },
      uFlowStrength: { value: flowStrength },
      uDecay: { value: decay },
      uFalloffStart: { value: falloffStart },
      uFogFallSpeed: { value: fogFallSpeed },
      uColor: { value: new THREE.Vector3(1, 1, 1) },
      uFade: { value: hasFadedRef.current ? 1 : 0 }
    };
    uniformsRef.current = uniforms;

    const material = new THREE.RawShaderMaterial({
      vertexShader: VERT,
      fragmentShader: FRAG,
      uniforms,
      transparent: false,
      depthTest: false,
      depthWrite: false,
      blending: THREE.NormalBlending
```

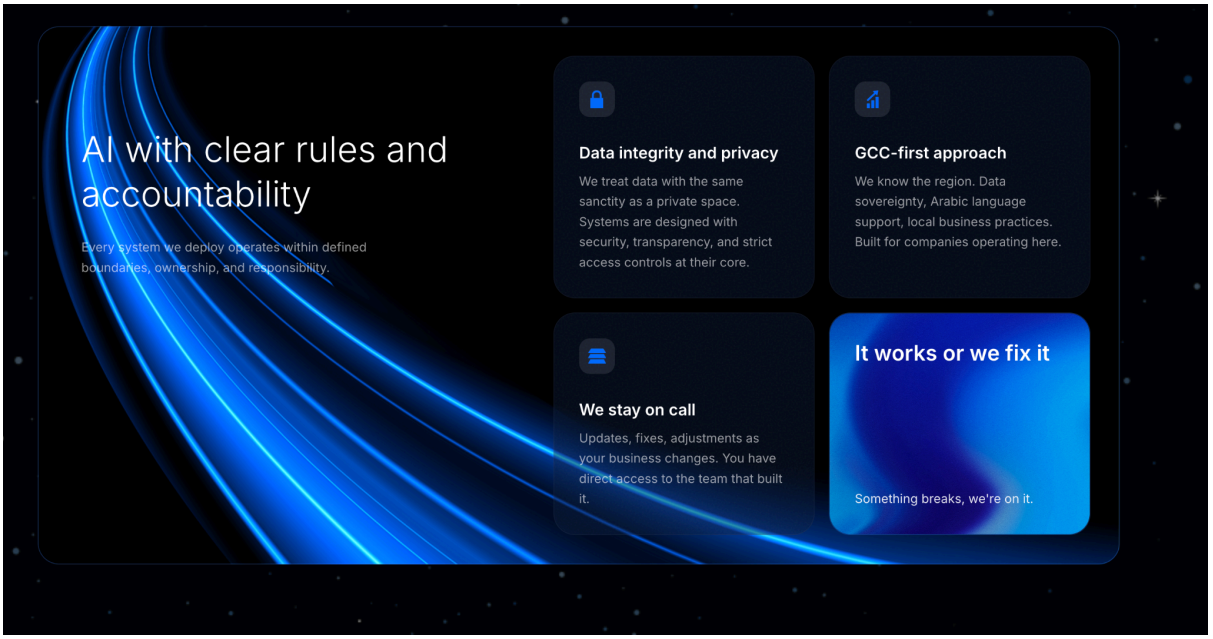
```
});

const mesh = new THREE.Mesh(geometry, material);
mesh.frustumCulled = false;
scene.add(mesh);

const clock = new THREE.Clock();
let prevTime = 0;
let fade = hasFadedRef.current ? 1 : 0;

const mouseTarget = new THREE.Vector2(0, 0);
const mouseSmooth = new THREE.Vector2(0, 0);

const setSizeNow = () => {
  const w = mount.clientWidth || 1;
  const h = mount.clientHeight || 1;
  const pr = currentDprRef.current;

  const last = lastSizeRef.current;
  const sizeChanged = Math.abs(w - last.width) > 0.5 || Math.abs(h - last.height) > 0.5;
  const dprChanged = Math.abs(pr - last.dpr) > 0.01;
  if (!sizeChanged && !dprChanged) {
    return;
  }

  lastSizeRef.current = { width: w, height: h, dpr: pr };
  renderer.setPixelRatio(pr);
  renderer.setSize(w, h, false);
  uniforms.iResolution.value.set(w * pr, h * pr, pr);
  rectRef.current = canvas.getBoundingClientRect();

  if (!pausedRef.current) {
    renderer.render(scene, camera);
  }
};

let resizeRaf = 0;
const scheduleResize = () => {
  if (resizeRaf) cancelAnimationFrame(resizeRaf);
  resizeRaf = requestAnimationFrame(setSizeNow);
};

setSizeNow();
const ro = new ResizeObserver(scheduleResize);
ro.observe(mount);

const io = new IntersectionObserver(
  entries => {
```

```
    inViewRef.current = entries[0]?.isIntersecting ?? true;
  },
  { root: null, threshold: 0 }
);
io.observe(mount);

const onVis = () => {
  pausedRef.current = document.hidden;
};
document.addEventListener('visibilitychange', onVis, { passive: true });

const updateMouse = (clientX, clientY) => {
  const rect = rectRef.current;
  if (!rect) return;
  const x = clientX - rect.left;
  const y = clientY - rect.top;
  const ratio = currentDprRef.current;
  const hb = rect.height * ratio;
  mouseTarget.set(x * ratio, hb - y * ratio);
};
const onMove = ev => updateMouse(ev.clientX, ev.clientY);
const onLeave = () => mouseTarget.set(0, 0);
canvas.addEventListener('pointermove', onMove, { passive: true });
canvas.addEventListener('pointerdown', onMove, { passive: true });
canvas.addEventListener('pointerenter', onMove, { passive: true });
canvas.addEventListener('pointerleave', onLeave, { passive: true });

const onCtxLost = e => {
  e.preventDefault();
  pausedRef.current = true;
};
const onCtxRestored = () => {
  pausedRef.current = false;
  scheduleResize();
};
canvas.addEventListener('webglcontextlost', onCtxLost, false);
canvas.addEventListener('webglcontextrestored', onCtxRestored, false);

let raf = 0;

const clamp = (v, lo, hi) => Math.max(lo, Math.min(hi, v));
const dprFloor = 0.6;
const lowerThresh = 50;
const upperThresh = 58;
let lastDprChangeRef = 0;
const dprChangeCooldown = 2000;

const adjustDprIfNeeded = now => {
```

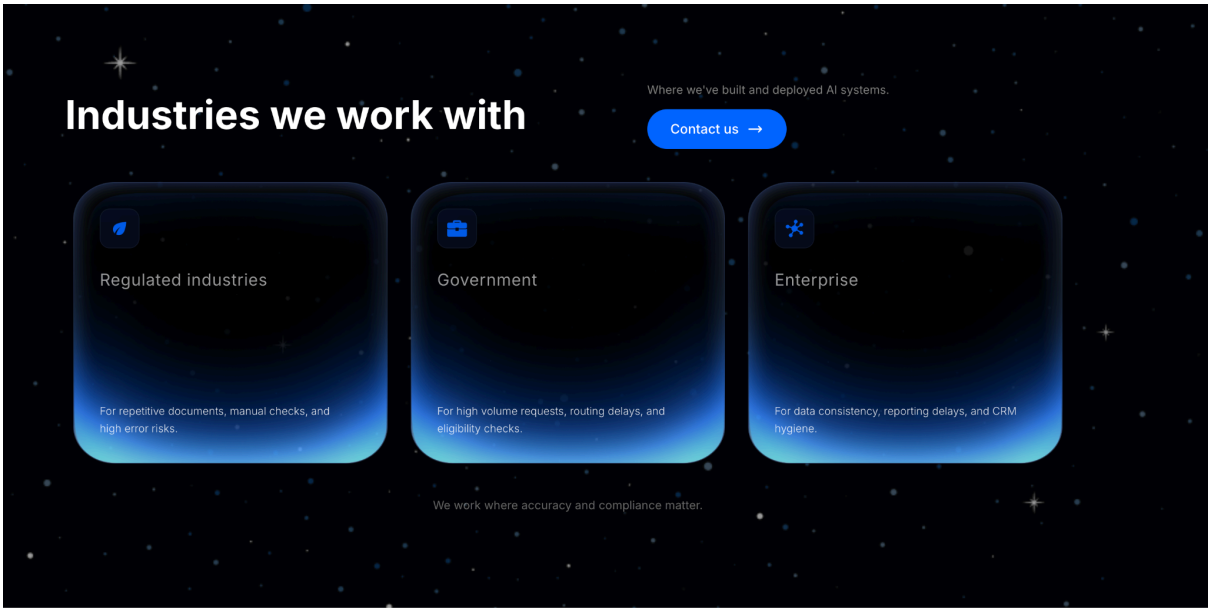
```
    const elapsed = now - lastFpsCheckRef.current;
    if (elapsed < 750) return;

    const samples = fpsSamplesRef.current;
    if (samples.length === 0) {
      lastFpsCheckRef.current = now;
      return;
    }
    const avgFps = samples.reduce((a, b) => a + b, 0) / samples.length;

    let next = currentDprRef.current;
    const base = baseDprRef.current;

    if (avgFps < lowerThresh) {
      next = clamp(currentDprRef.current * 0.85, dprFloor, base);
    } else if (avgFps > upperThresh && currentDprRef.current < base) {
      next = clamp(currentDprRef.current * 1.1, dprFloor, base);
    }

    if (Math.abs(next - currentDprRef.current) > 0.01 && now - lastDprChangeRef >
dprChangeCooldown) {
      currentDprRef.current = next;
      lastDprChangeRef = now;
      setSizeNow();
    }

    fpsSamplesRef.current = [];
    lastFpsCheckRef.current = now;
  };

  const animate = () => {
    raf = requestAnimationFrame(animate);
    if (pausedRef.current || !inViewRef.current) return;

    const t = clock.getElapsedTime();
    const dt = Math.max(0, t - prevTime);
    prevTime = t;

    const dtMs = dt * 1000;
    emaDtRef.current = emaDtRef.current * 0.9 + dtMs * 0.1;
    const instFps = 1000 / Math.max(1, emaDtRef.current);
    fpsSamplesRef.current.push(instFps);

    uniforms.iTime.value = t;

    const cdt = Math.min(0.033, Math.max(0.001, dt));
    uniforms.uFlowTime.value += cdt;
    uniforms.uFogTime.value += cdt;
```

```
    if (!hasFadedRef.current) {
      const fadeDur = 1.0;
      fade = Math.min(1, fade + cdt / fadeDur);
      uniforms.uFade.value = fade;
      if (fade >= 1) hasFadedRef.current = true;
    }

    const tau = Math.max(1e-3, mouseSmoothTime);
    const alpha = 1 - Math.exp(-cdt / tau);
    mouseSmooth.lerp(mouseTarget, alpha);
    uniforms.iMouse.value.set(mouseSmooth.x, mouseSmooth.y, 0, 0);

    renderer.render(scene, camera);

    adjustDprIfNeeded(performance.now());
  };

  animate();

  return () => {
    cancelAnimationFrame(raf);
    ro.disconnect();
    io.disconnect();
    document.removeEventListener('visibilitychange', onVis);
    canvas.removeEventListener('pointermove', onMove);
    canvas.removeEventListener('pointerdown', onMove);
    canvas.removeEventListener('pointerenter', onMove);
    canvas.removeEventListener('pointerleave', onLeave);
    canvas.removeEventListener('webglcontextlost', onCtxLost);
    canvas.removeEventListener('webglcontextrestored', onCtxRestored);
    geometry.dispose();
    material.dispose();
    renderer.dispose();
    if (mount.contains(canvas)) mount.removeChild(canvas);
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [dpr]);

useEffect(() => {
  const uniforms = uniformsRef.current;
  if (!uniforms) return;

  uniforms.uWispDensity.value = wispDensity;
  uniforms.uTiltScale.value = mouseTiltStrength;
  uniforms.uBeamXFrac.value = horizontalBeamOffset;
  uniforms.uBeamYFrac.value = verticalBeamOffset;
  uniforms.uFlowSpeed.value = flowSpeed;
```

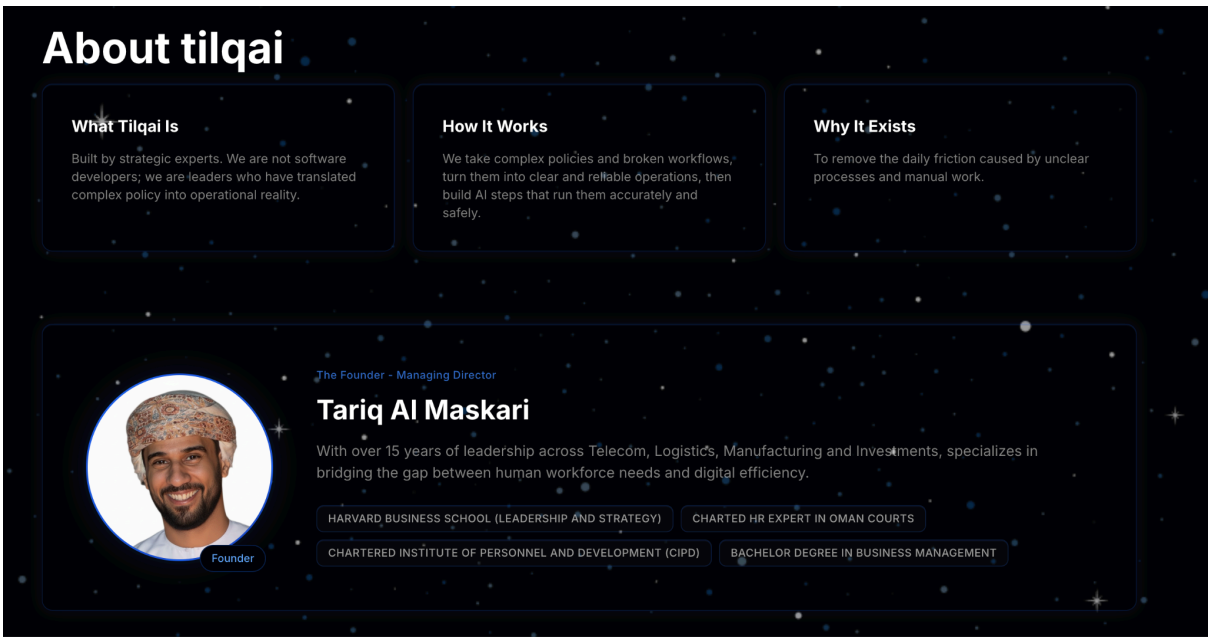
```
    uniforms.uVLenFactor.value = verticalSizing;
    uniforms.uHLenFactor.value = horizontalSizing;
    uniforms.uFogIntensity.value = fogIntensity;
    uniforms.uFogScale.value = fogScale;
    uniforms.uWSpeed.value = wispSpeed;
    uniforms.uWIntensity.value = wispIntensity;
    uniforms.uFlowStrength.value = flowStrength;
    uniforms.uDecay.value = decay;
    uniforms.uFalloffStart.value = falloffStart;
    uniforms.uFogFallSpeed.value = fogFallSpeed;

    const { r, g, b } = hexToRGB(color || '#FFFFFF');
    uniforms.uColor.value.set(r, g, b);
  }, [
    wispDensity,
    mouseTiltStrength,
    horizontalBeamOffset,
    verticalBeamOffset,
    flowSpeed,
    verticalSizing,
    horizontalSizing,
    fogIntensity,
    fogScale,
    wispSpeed,
    wispIntensity,
    flowStrength,
    decay,
    falloffStart,
    fogFallSpeed,
    color
  ]);

  return <div ref={mountRef} className={`laser-flow-container ${className || ''}`}
style={style} />;
};

export default LaserFlow;


.laser-flow-container {
  width: 100%;
  height: 100%;
  position: relative;
  pointer-events: none;
}
```



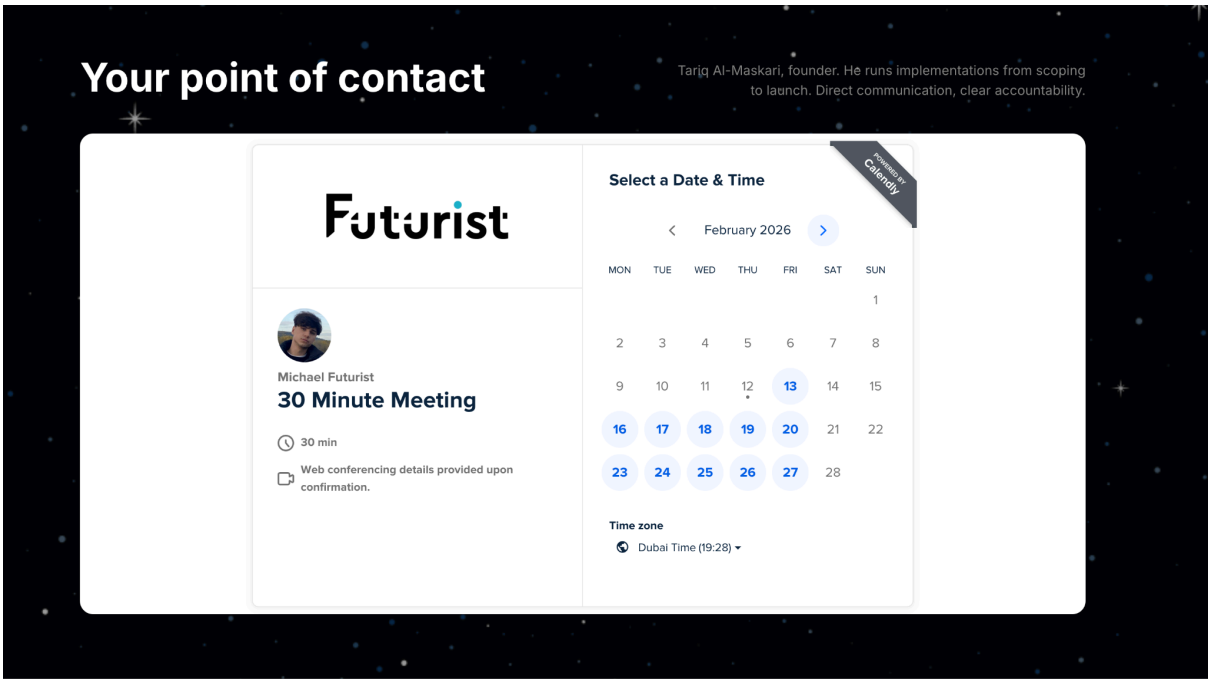
немного оживить блоки. сейчас статично выглядят. при наведении должны немного поворачиваться как 3д плитки



иконки нужно покрупнее - сейчас слишком пусто выглядят блоки. много пустого пространства



как то по другому скомпоновать + лого фирменный. сейчас выглядит дешево - 3 плитки верхние.



тут белые края лишние по бокам виджета - нужно убрать их